执行：能力实现详解

- 来源: https://class.imooc.com/lesson/5290#mid=105150
- 阶段: 阶段六: 【智能体】从单体到多体
- 周: 第12周 Multi-Agent：多智能体协作
- 模块: Agent的核心架构剖析
- 课节: 图文 1-2 执行：能力实现详解

1-2 执行：能力实现详解

执行：能力实现详解

> [!TIP] 阅读指南 上一节我们从宏观视角了解了Agent的整体架构: 循环流程:Perceive → Think → Act → Observe 模块分工:感知层、规划层、推理层、行动层、记忆系统 两者的映射关系 但知道架构还不够。就像你看了大脑的解剖图，知道额叶负责决策、海马体负责记忆，但这些区域具体是怎么工作的?本节我们就来深入三大核心能力: 感知层:如何将混乱的输入(用户说话、工具返回、环境信息)整合成LLM可理解的统一上下文? 规划层:如何将高层目标(“帮我写周报”)拆解为可执行的子任务序列? 行动层:如何准确执行规划的每个步骤，调用工具并处理结果?

感知层：从混乱到清晰

感知层的核心问题

第3节我们已经知道，感知层负责将用户输入、工具返回、环境状态、历史记忆这四类输入整合成LLM可理解的文本。

但这里有个关键问题:为什么要整合和组装，而不是直接解析?

为什么要整合和组装，而不是直接解析?

真实场景：天气查询 Agent 的感知过程

假设用户说：“明天杭州天气怎么样?”

传统程序的处理方式

```
# 硬编码的规则
def parse_weather_query(text):
    if "明天" in text:
        date = get_tomorrow()
    if "杭州" in text:
        city = "杭州"

    return get_weather(city, date)
```

传统程序有几个问题：

- 只能处理固定模板
- 无法理解"后天"、 “大后天”
- 无法处理"西湖那边"、"我老家"等模糊表达

Agent 的感知方式：上下文组装

Agent 不直接解析，而是将所有相关信息组装成一个完整的上下文,交给 LLM 理解:

```
def perceive(user_input, context):
    """
    感知层:组装完整的上下文信息
    """
    # ▸ 第 1 步:收集系统信息
    system_info = {
        "current_time": "2024年12月9日 14:30",
        "user_location": "上海"  # 从用户配置中获取
    }

    # ▸ 第 2 步:检索历史记忆
    history = context.get("conversation_history", [])

    # ▸ 第 3 步:列出可用工具
    available_tools = [
        {
            "name": "get_weather",
            "description": "获取指定城市和日期的天气信息",
            "parameters": {
                "city": "城市名称",
                "date": "日期(格式:YYYY-MM-DD)"
            }
        }
    ]

    # ▸ 第 4 步:组装成 LLM 可理解的 Prompt
    prompt = f"""
## 系统信息
当前时间: {system_info['current_time']}
用户位置: {system_info['user_location']}

## 对话历史
{format_history(history)}

## 可用工具
{format_tools(available_tools)}

## 用户请求
{user_input}

请理解用户意图,决定是否需要调用工具。
"""
```

```
    return prompt
```

感知层做的是信息整合,而不是意图识别。

信息整合

意图识别

- 传统程序:先识别意图(“这是天气查询”),再调用功能

传统程序

- Agent:把所有信息交给 LLM,让 LLM 自己理解和决策

这就是为什么 Agent 能处理复杂、模糊的输入——理解能力来自 LLM,而不是硬编码的规则。

多模态感知

现在，Agent 的感知能力扩展到了多模态:

```
用户上传一张图片 + 文字:"这是什么动物?"

       多模态感知层的处理流程
═══════════════════════════════════
1. 图像 → 通过 Qwen/Gemini 理解
2. 文字 → 提取用户意图
3. 组合 → 形成统一的上下文
4. 输出 → 交给推理层决策
```

示例代码:

示例代码

```
def multimodal_perceive(image, text, context):
    """
    多模态感知:处理图像 + 文字输入
    """
    # ▸ 调用视觉模型理解图像
    image_description = vision_model.analyze(image)
    # 返回: "图片中是一只橘色的猫,正在睡觉"

    # ▸ 组装完整上下文
    prompt = f"""
## 图像理解
{image_description}

## 用户问题
{text}

请回答用户的问题。
"""

    return prompt
```

这种方式的优势:无需为每种输入类型单独编写解析逻辑,LLM 自动理解多模态信息。

规划层：从目标到路径

规划能力的核心来源：LLM的推理

第3节我们知道了规划层负责将高层目标拆解为子任务序列。但一个关键问题被忽略了——谁在做规划，规划能力从哪来？答案是：LLM本身。

Agent的规划能力不是预先编写的算法，而是通过Prompt引导LLM进行任务拆解。

```
# 传统程序:硬编码的规划逻辑
def plan_travel():
    steps = [
        "查询天气",
        "订票",
        "订酒店"
    ]
    return steps

# Agent:让LLM自己规划
def plan_with_llm(goal, context):
    prompt = f"""
你是一个任务规划专家。

用户目标:{goal}
可用工具:{context['tools']}

请将目标拆解为具体的执行步骤,每步说明:
1. 要做什么
2. 调用哪个工具
3. 预期结果

以JSON格式返回。
"""

    plan = llm.generate(prompt)
    return parse_json(plan)
```

```
规划不是算法,而是LLM的能力
═══════════════════════════════════════
```

- 传统程序:if-else预设所有可能路径
- Agent:LLM根据目标动态生成执行计划

```
为什么LLM能规划?
→ 训练数据中见过大量任务分解案例
→ 理解任务之间的依赖关系
→ 能推理"先做什么,后做什么"
```

两种规划方式的LLM实现对比

第3节提到了Plan-then-Execute和ReAct两种策略。但它们的本质区别在于给LLM的Prompt不同:

Plan-then-Execute：一次性生成完整计划

```
def plan_then_execute(goal, context):
    # ▸ 关键:Prompt要求LLM"一次性生成全部步骤"
    prompt = f"""
目标:{goal}

请生成**完整的执行计划**,列出所有步骤:
- 步骤1: [具体操作]
- 步骤2: [具体操作]
- 步骤3: [具体操作]

计划生成后,按顺序执行即可。
"""

    plan = llm.generate(prompt)  # LLM输出完整计划

    # 按计划执行
    for step in plan:
        execute(step)
```

LLM的角色:一次性推理出所有步骤，类似于"先想好再行动"。

LLM的角色

ReAct：每步都让LLM重新推理

```
def react_planning(goal):
    state = {"goal": goal, "done": False, "history": []}

    while not state['done']:
        # ▸ 关键：每步都重新调用LLM
        prompt = f"""
目标：{state['goal']}
已执行：{state['history']}
请决定下一步行动...
"""
        response = llm.generate(prompt)

        if response.need_tool:
            result = execute_tool(response.tool, response.params)
            state['history'].append(result)
        else:
            return response.answer
```

LLM的角色:每一步都基于最新情况重新推理，类似于“边做边想”。

混合规划：让LLM自己选择策略

第3节讲了混合规划的分层思想。但更高级的做法是:让LLM自己决定用哪种策略。

让LLM自己决定用哪种策略

```
class HybridPlanner:
    """
    混合规划器:根据任务类型选择策略
    """
    def plan(self, task, context):
        # ▸ 第 1 层:规则引擎。如果能自己判断，就不用麻烦LLM了，浪费钱
        if self.rule_engine.can_handle(task):
            return self.rule_engine.execute(task)

        # ▸ 第 2 层:判断任务复杂度。不再手写规则内的，问LLM让他判断
        complexity = self.llm.classify_task(task)

        if complexity == "simple":
            # 一次性规划
            return self.plan_then_execute(task, context)
        else:
            # ReAct 动态规划
            return self.react_planning(task, context)
```

真实案例：LangGraph 的规划实现

第9章我们详细学习了 LangGraph 框架。现在回过头来看，LangGraph 其实就是将我们刚才讨论的两种规划策略通过状态图的方式实现了:

```
LangGraph 如何体现规划能力?
═══════════════════════════════════════
```

- 节点(Node):封装 LLM 推理
  - 每个节点都可以调用 LLM
  - 让 LLM 决定下一步做什么

- 边(Edge):实现两种规划
  - 固定边 → Plan-then-Execute
  - 条件边 → ReAct(动态规划)

- 状态(State):传递规划上下文
  - 保存中间结果
  - LLM 根据历史状态调整策略

LangGraph 的**条件边(conditional_edges)**就是让 LLM 在执行过程中重新规划:

```
# 回顾第9章的条件分支示例
def should_continue(state):
    """
    关键:这里调用 LLM 判断是否继续
    → 这就是 ReAct 的动态规划
    """
    decision = llm.analyze(state['current_result'])

    if decision == "需要更多信息":
        return "continue"  # 继续执行
    else:
        return "finish"    # 完成任务

# 添加条件边
workflow.add_conditional_edges(
    "analyze",
    should_continue,  # ← LLM 每次都重新决策
    {
        "continue": "retrieve_more",
        "finish": "generate_answer"
    }
)
```

与本节规划策略的对应:

与本节规划策略的对应

```
规划策略 → LangGraph 实现
═══════════════════════════════════════

Plan-then-Execute:
● 用固定的 add_edge() 串联节点
● LLM 只在起点生成完整计划

ReAct:
● 用 add_conditional_edges()
● 每个节点都可以让 LLM 重新决策

混合规划:
● 简单任务:固定流程(add_edge)
● 复杂任务:动态判断(conditional_edges)
```

行动层：从计划到执行

前面我们讲了感知如何整合信息、规划如何拆解任务。但有了计划后，谁来执行?

谁来执行?

答案就是:行动层(Action Layer)。

行动层(Action Layer)

行动层的核心职责

行动层是 Agent 与外部世界的接口，它负责:

与规划层的区别:

与规划层的区别

- 规划层:决定“调用哪个工具”和“传什么参数”（思考）
- 行动层:真正去执行这个调用（行动）

```
class ActionLayer:
    """
    行动层:执行工具调用
    """
    def execute(self, tool_call):
        # ▸ 第 1 步:校验工具是否存在
        if tool_name not in self.tools:
            return error("工具不存在")

        # ▸ 第 2 步:校验参数（类型、必需性）
        validated_params = validate(tool_params)

        # ▸ 第 3 步:执行工具
        try:
            result = tool.run(**validated_params)
            return success(result)
        except Exception as e:
            # ▸ 第 4 步:错误处理与重试
            return handle_error(e)
```

三大能力的协同：完整闭环

现在我们已经学习了感知、规划、行动三大能力。但它们如何协同工作，形成闭环？

它们如何协同工作，形成闭环？

闭环工作流程

- 感知层每次都会包含上一轮的 Observation
- 规划层根据最新信息重新决策
- 行动层执行后立即返回结果
- 循环直到 LLM 判断任务完成

完整示例：日志分析 Agent

这个 Agent 接收应用的错误日志，自动诊断问题根因并给出修复建议。例如收到一段数据库连接失败的日志，它会先感知日志内容，规划排查步骤（检查连接配置 → 验证网络 → 分析错误码），然后调用工具去读取相关配置文件和运行状态，最后基于收集到的信息给出结论。

核心实现：

核心实现

```
class LogAnalysisAgent:
    def run(self, user_input):
        """主循环：感知 → 规划 → 行动 → 观察"""
        for i in range(max_iterations):
            # ▸ 第 1 步：感知（整合上下文）
            context = self._perceive(user_input)

            # ▸ 第 2 步：规划（LLM 决策）
            decision = self._plan(context)

            # ▸ 第 3 步：判断是否需要行动
            if decision['type'] == 'final_answer':
                return decision['answer']

            # ▸ 第 4 步：行动（执行工具）
            observation = self._act(decision)

            # ▸ 第 5 步：记录到历史
            self.history.append({...})
```

完整代码：详见 samples/chapter11/log_agent/ 目录：

```
log_agent/
├── log_agent.py          # 完整的 Agent 实现（感知层、规划层、行动层）
├── sample_logs/          # 示例日志文件
│   ├── app_error.log     #   - 数据库连接错误示例
│   └── API_error.log     #   - API 验证错误示例
└── README.md             # 详细的运行指南和代码讲解
```

下节预告

现在你已经完整了解了 Agent 的三大核心能力——感知如何整合信息、规划如何拆解任务、行动如何执行工具。

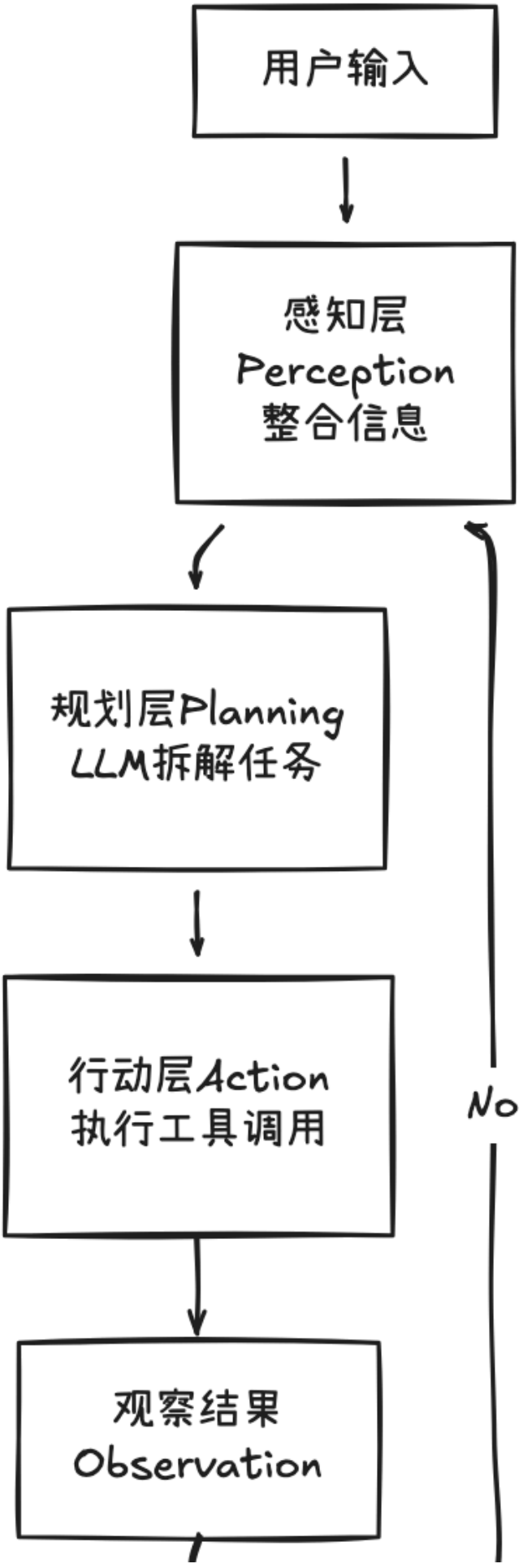
但问题来了：Agent 如何"记住"过去的经验？ 如果每次都从零开始，Agent 就无法积累知识、学习经验。

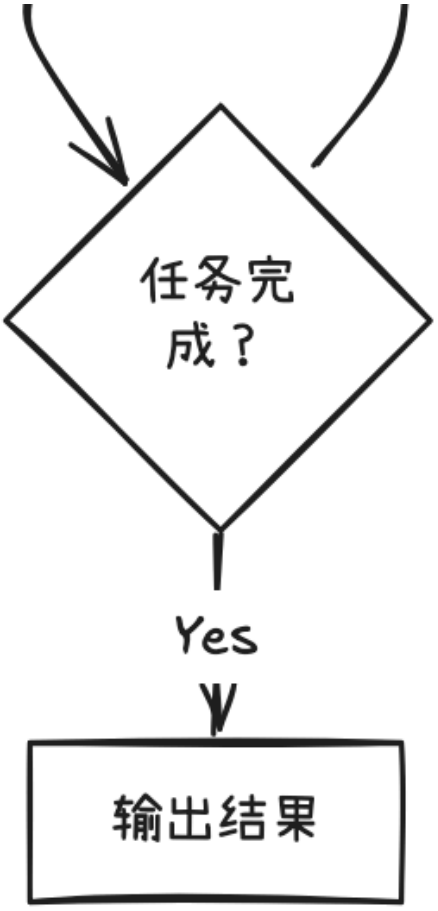
下一节，我们将深入探讨 Agent 的记忆系统——从短期记忆到长期记忆，从工作记忆到情景记忆，你会看到，记忆系统如何让 Agent 从"工具"升级为"助手"。

■ 学点英语

中文	English	音标	说明
上下文组装	Context Assembly	/ˈkɒntekst əˈsembli/	感知层将多源信息整合为完整Prompt的过程
先规划后执行	Plan-then-Execute	/plæn ðen ˈeksɪkjuːt/	先生成完整计划再逐步执行的规划策略
交错规划	Interleaved Planning	/ˌɪntərˈliːvd ˈplænɪŋ/	每执行一步后根据结果重新规划的ReAct策略
混合规划	Hybrid Planning	/ˈhaɪbrɪd ˈplænɪŋ/	规则引擎+LLM分层结合，平衡效率与灵活性
行动层	Action Layer	/ˈækʃn ˈleɪər/	将LLM决策转化为实际工具调用的执行模块
多模态感知	Multimodal Perception	/ˌmʌltiˈmoʊdl pərˈsepʃn/	Agent处理图像、语音等非文本输入的能力

图片


用户输入
感知层
Perception
整合信息
规划层Planning
LLM拆解任务
行动层Action
执行工具调用
观察结果
Observation
No


任务完成？
Yes
输出结果